Programming Assignment 3 — Poucos rótulos: fauna em câmeras trap com dados não rotulados

Disciplina	Aprendizado Profundo
Professor	Dario Oliveira
Monitor	Erick Brito
Entrega	16/11, 23h59 — B41254@fgv.edu.br
Apresentação	18/11 e 20/11 - **Podem ser alterados ainda** (horários a definir)
Formato	Duplas

1. O problema

Câmeras trap são câmeras com sensor de movimento que ficam semanas no mato fotografando o que passa na frente delas. Quando alguém volta para buscar o cartão de memória, encontra milhares de imagens e ninguém para dizer qual bicho está em cada uma. Em 2025 a Tryolabs organizou no Kaggle o *Cup-ybara*, um desafio com vídeos de câmeras trap da fauna uruguaia para uma ONG local, a AMBÁ. A regra que deixava o desafio interessante era que o conjunto de treino não tinha rótulo nenhum. É a situação normal de quem faz monitoramento: muito dado e pouquíssima gente para anotar.

No laboratório de dados não rotulados vocês viram três respostas para isso no MNIST: Deep k-Means, SimCLR com avaliação linear e FixMatch com 100 rótulos. Também viram o SwAV no CIFAR-10. No MNIST tudo isso funciona bem, porque o dígito ocupa a imagem inteira, o fundo é preto e as dez classes têm o mesmo tamanho. Numa câmera trap, nada disso vale:

- em um quarto das imagens, o animal ocupa menos de 2% do quadro;
- muitas fotos são noturnas, em infravermelho, sem cor;
- a mesma câmera fotografa sempre o mesmo fundo, e o fundo é um atalho tentador;
- as classes são desbalanceadas: no nosso pool há 690 imagens de urso e 45 de cavalo.

A tarefa é classificar a espécie em cada imagem tendo **5 rótulos por classe** e alguns milhares de imagens sem rótulo. Vocês escolhem como usar as imagens sem rótulo, e o produto final precisa ser um modelo **pequeno, que roda em CPU**, como seria um modelo levado para o campo.

2. Dados

ENA24-detection (LILA BC, licença CDLA-Permissive-1.0). Imagens de câmeras trap do leste da América do Norte, uma espécie por imagem, 21 classes. O pacote já está preparado, e é só ele que vocês usam:

Dados PA3 (`pa3_dados.zip`, 415 MB)

`https://drive.google.com/file/d/1g7FGqtXOk_ziNKBwQAk8BxjPOaPCnbui/view?usp=sharing`

Conjunto	Imagens	Rótulo
`train`	6.505	Não, exceto as imagens listadas nos arquivos de orçamento
`rotulos_1/5/20.csv`	21 / 105 / 420	1, 5 ou 20 por classe, sempre subconjuntos do `train`, aninhados
`val`	202	Sim. Até 10 por classe
`test_public`	811	Sim
`test_private`	∼800	Fica com o monitor

- As imagens vêm reduzidas para 512 pixels no lado maior, sem a faixa de texto que a câmera imprime (data, hora e nome da câmera), e com nomes trocados por um código. Por causa do corte, as proporções variam de uma câmera para outra.
- O **orçamento padrão é o de 5 rótulos por classe**. Os de 1 e 20 servem para a Parte 2.
- O `val` é pequeno de propósito. Com 5 rótulos por classe, ninguém no mundo real teria mil imagens rotuladas para escolher hiperparâmetros. É nele que vocês fazem seleção de modelo.
- O `test_public` serve só para reportar números. Usá-lo para escolher época, limiar ou arquitetura é sobreajuste ao teste, e a diferença aparece no `test_private`.
- Cavalo tem só 45 imagens no treino, 2 no `val` e 4 no `test_public`. Com o orçamento de 20 por classe, quase metade fica rotulada, e o F1 de cavalo no teste muda bastante com um acerto a mais ou a menos. Isso faz parte do problema.

Split. O ENA24 não traz local nem horário como metadado, mas as câmeras imprimem o próprio nome, a data e a hora numa faixa da imagem. Essa faixa foi lida e agrupada considerando imagens por câmera e noite (do meio-dia ao meio-dia seguinte). Cada grupo vai inteiro para um único conjunto, então uma rajada ou uma visita do mesmo animal nunca aparece no treino e no teste ao mesmo tempo. A mesma câmera pode aparecer em conjuntos diferentes, em noites diferentes: o fundo se repete, e aprender o fundo em vez do animal é o atalho que a Parte 3 investiga. Os rótulos do orçamento e o `val` também foram escolhidos em grupos distintos: os 5 rótulos de uma classe vêm de 5 câmeras-noite diferentes. Depois disso, a faixa foi cortada das imagens.

3. Regras de engenharia

Permitido: PyTorch, torchvision, timm; backbones com pesos de ImageNet (classificação supervisionada) para o modelo de vocês; albumentations; sklearn; o código que vocês escreveram nos laboratórios (NT-Xent, perda do FixMatch, SwAV).
Permitido com condição: *foundation models*, isto é, CLIP, SigLIP, BioCLIP e qualquer modelo treinado em dados de espécies ou de câmeras trap (o que inclui MegaDetector e SpeciesNet). Só na Trilha C, só com pesos abertos rodando no Colab ou na máquina de vocês, e só como **professor**: ele pode olhar o `train` e o `test_public` (para vocês medirem o professor), nunca o `val` durante o treino, e não pode fazer parte do modelo entregue.
Proibido:

- rotular imagens à mão. Os únicos rótulos humanos são os do pacote;
- API de LLM ou de modelo fechado (GPT, Gemini, Claude e afins) para rotular ou classificar;
- baixar o ENA24 original ou qualquer outro dado externo, rotulado ou não;
- treinar ou fazer seleção de modelo no `test_public`.

O modelo entregue tem no máximo **12 milhões de parâmetros**, contados com `sum(p.numel() for p in model.parameters())`, o que comporta ResNet-18, MobileNetV3 e EfficientNet-B0. Ele roda em CPU, sem internet, e classifica o `test_private` em menos de 10 minutos numa máquina comum.

Parte 0 — O piso

1. Treinem um classificador supervisionado só com `rotulos_5.csv` (105 imagens), em duas versões: (a) uma CNN pequena treinada do zero; (b) o backbone que vocês vão usar no resto do PA, partindo do ImageNet, com *fine-tuning*.
2. Reportem **F1 macro** e acurácia no `test_public`, mais a matriz de confusão da versão (b). A métrica principal do PA é o F1 macro, que pesa cavalo igual a urso; acurácia sozinha esconde as classes raras.

3. Mostrem a curva de treino e validação da versão (b). Com 105 imagens, em que época ela começa a decorar?

O teto é o F1 de um modelo treinado com *todos* os rótulos do `train`. O monitor treinou uma ResNet-18 partindo do ImageNet, com *fine-tuning* e época escolhida no `val`, e obteve **0,686 de F1 macro** no `test_public`. Usem esse número como a linha do teto na Parte 2, mas tratem-no como referência, não como limite: vocês pode chegar perto dele ou até passá-lo. A Parte 1 é sobre quanto da distância entre o piso e o teto dá para recuperar sem pedir mais rótulos.

Parte 1 — Usando o que não tem rótulo (escolham UMA trilha)

Todas as trilhas usam o mesmo backbone da Parte 0 (b) e o orçamento de 5 por classe, e se comparam com o piso nas mesmas métricas.

Trilha A — Auto-supervisionado

Continuem o pré-treino do backbone ImageNet com SimCLR ou SwAV sobre as 6.505 imagens do `train`. Depois façam avaliação linear e *fine-tuning* com os 105 rótulos.

- Obrigatório: avaliação linear com três backbones lado a lado: ImageNet puro, ImageNet + auto-supervisionado, e aleatório + auto-supervisionado. O pré-treino no domínio vale alguma coisa em cima do ImageNet?
- Obrigatório: um experimento sobre aumentação. Um *random resized crop* numa imagem em que o animal ocupa 2% do quadro gera, na maioria das vezes, um par positivo sem animal nenhum. O que a rede aprende a aproximar nesse caso? Escolham uma mudança na aumentação, meçam e expliquem.

Trilha B — Semi-supervisionado

FixMatch, como no laboratório, com os 105 rótulos e o `train` inteiro como não rotulado.

- Obrigatório: precisão e cobertura dos pseudo-rótulos **por classe**, em função do limiar τ. Meçam isso rodando o pseudo-rotulador sobre o `test_public` como se ele não tivesse rótulo. É só diagnóstico: a escolha de τ é feita no `val`.
- Obrigatório: com classes desbalanceadas, os pseudo-rótulos tendem a ir para as classes grandes, e o modelo treinado neles fica ainda mais enviesado. Mostrem se isso acontece no seu treino e testem uma correção (limiar por classe, alinhamento de distribuição ou outra que vocês justifiquem).

Trilha C — Professor e aluno

Um *foundation model* aberto rotula o `train` e um modelo pequeno (o aluno) é treinado nesses pseudo-rótulos, junto com os 105 rótulos verdadeiros.

- Obrigatório: o F1 *zero-shot* do professor no `test_public`, com os nomes das classes como *prompt*. Esse número é uma linha de referência, igual ao piso.
- Obrigatório: a matriz de confusão do professor é a matriz de ruído dos pseudo-rótulos. Mostrem-na e digam onde o professor erra de forma sistemática (noite, animais pequenos, classes parecidas como *bird* e *american_crow*).
- Obrigatório: tratem esse ruído de alguma forma (filtro por confiança, rótulos suaves com temperatura, correção de rótulo como no SELFIE, ou um professor melhorado com os 105 rótulos) e respondam: em quais classes o aluno supera o professor, e por quê?

Uma pergunta para responder na apresentação (sem implementar). Se o modelo de vocês fosse instalado numa câmera no Uruguai, como no Cup-ybara, a primeira capivara que passasse

seria classificada como uma das 21 classes, com alguma confiança. O que no método de vocês daria um sinal de que aquela imagem não pertence a nenhuma das classes?

Parte 2 — A curva de orçamento

1. Rodem o piso (Parte 0 b) e o método da sua trilha com os três orçamentos: 1, 5 e 20 rótulos por classe. No orçamento de 5, usem 3 seeds e reportem média ± desvio. Nos outros, uma seed basta.
2. O gráfico obrigatório: F1 macro no `test_public` contra o número de rótulos (eixo x em escala log), uma curva para o piso e outra para o método, com a linha do teto. Na Trilha C, incluam também o professor.
3. Quebrem o ganho por classe no orçamento de 5: quais classes ganharam mais e quais não ganharam nada?

Parte 3 — Galeria de falhas e atalhos

Três erros do modelo final, cada um com:

- a imagem, a classe verdadeira, a prevista e a confiança;
- um mapa de saliência (Grad-CAM ou oclusão) que mostre para onde o modelo olhou;
- um diagnóstico concreto, por exemplo: "foto noturna, o animal ocupa 40×30 pixels depois do redimensionamento para 224, e o mapa está todo no tronco da árvore ao fundo".

Façam também a pergunta inversa: escolham 5 *acertos* e mostrem o mapa de saliência. O modelo acertou olhando para o animal ou para o fundo?

4. Entregáveis

Arquivo	Descrição
Repositório Git	Repositório do trabalho
`README.md`	Ambiente, um comando que treina, um comando que avalia
`predict.py`	`python predict.py -imagens PASTA -saida preds.csv`. Escreve um CSV com colunas `image_id,label`, usando os nomes de `classes.csv`. Roda em CPU, sem internet, e imprime o número de parâmetros do modelo
Pesos	Do modelo entregue (link se for grande)
`curva_orcamento.csv`	Os números da Parte 2: método, orçamento, seed, F1 macro, acurácia
`AI_LOG.md`	Ver abaixo

O monitor `predict.py` de vocês no `test_private`. Isso não é uma competição: o número privado serve para conferir que o modelo entregue é o que foi apresentado e que ele não foi ajustado ao `test_public`.

Não há relatório escrito. A avaliação é a apresentação. O repositório existe para dar lastro ao que vocês afirmarem lá: toda tabela, curva e imagem mostrada na apresentação tem que ser reproduzível a partir dele.

5. Política de uso de IA

Uso de IA é permitido e esperado. O que não é permitido é entregar algo que vocês não entendem.

Subam também um arquivo `AI_LOG.md` que descreve como usaram a IA nesse assignment. Podem citar alguns episódios e como recorreram às ferramentas de IA para resolvê-los.